

Статья Ретрансляция NTLM, Часть 2

 codeby.net/threads/retranslyatsiya-ntlm-chast-2.73923

Сергей Попов

June 17, 2020

Данная статья является переводом. Оригинал вот [тут](#)

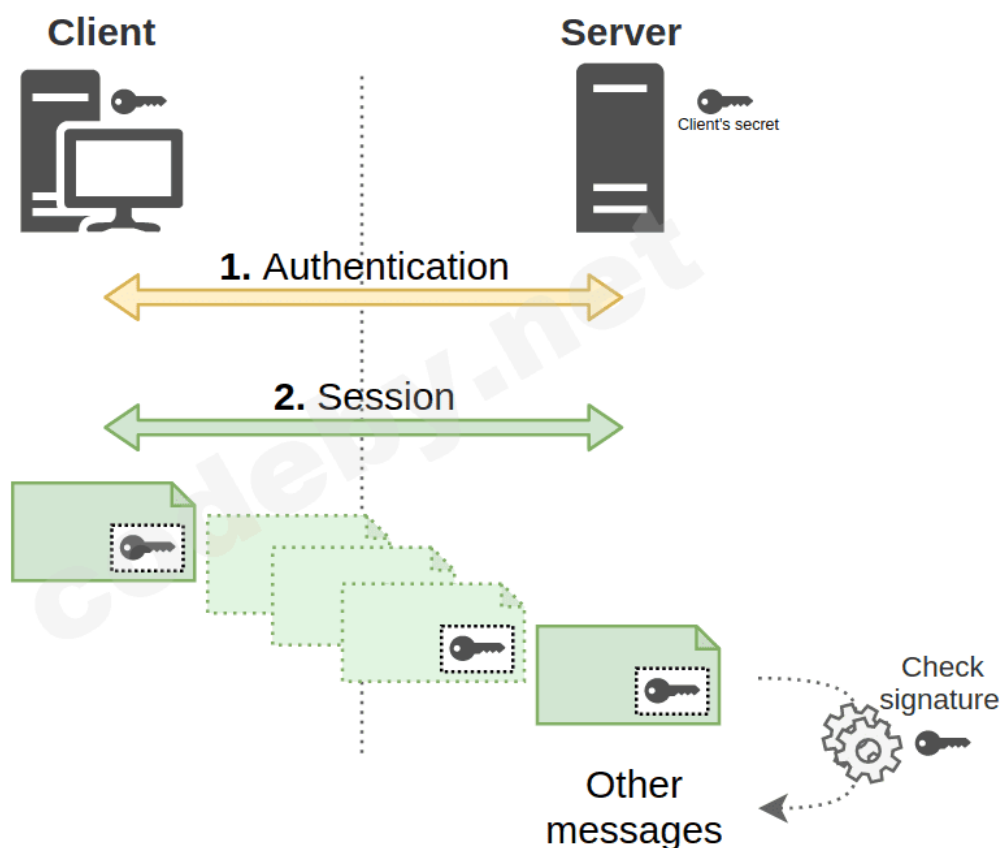
[Первая часть "Ретрансляция NTLM"](#)

Подпись сеанса

Принцип

Подпись является методом проверки подлинности и гарантирует, что информация не была подделана между отправкой и получением. Например, если пользователь jdoe посылает текст, I love hackndo, и подписывает этот документ в цифровом виде, то любой, кто получит этот документ и его подпись. Также, можно проверить, что редактировал его jdoe, и jdoe написал это предложение, а никто другой. Подпись гарантирует, что документ не был изменен.

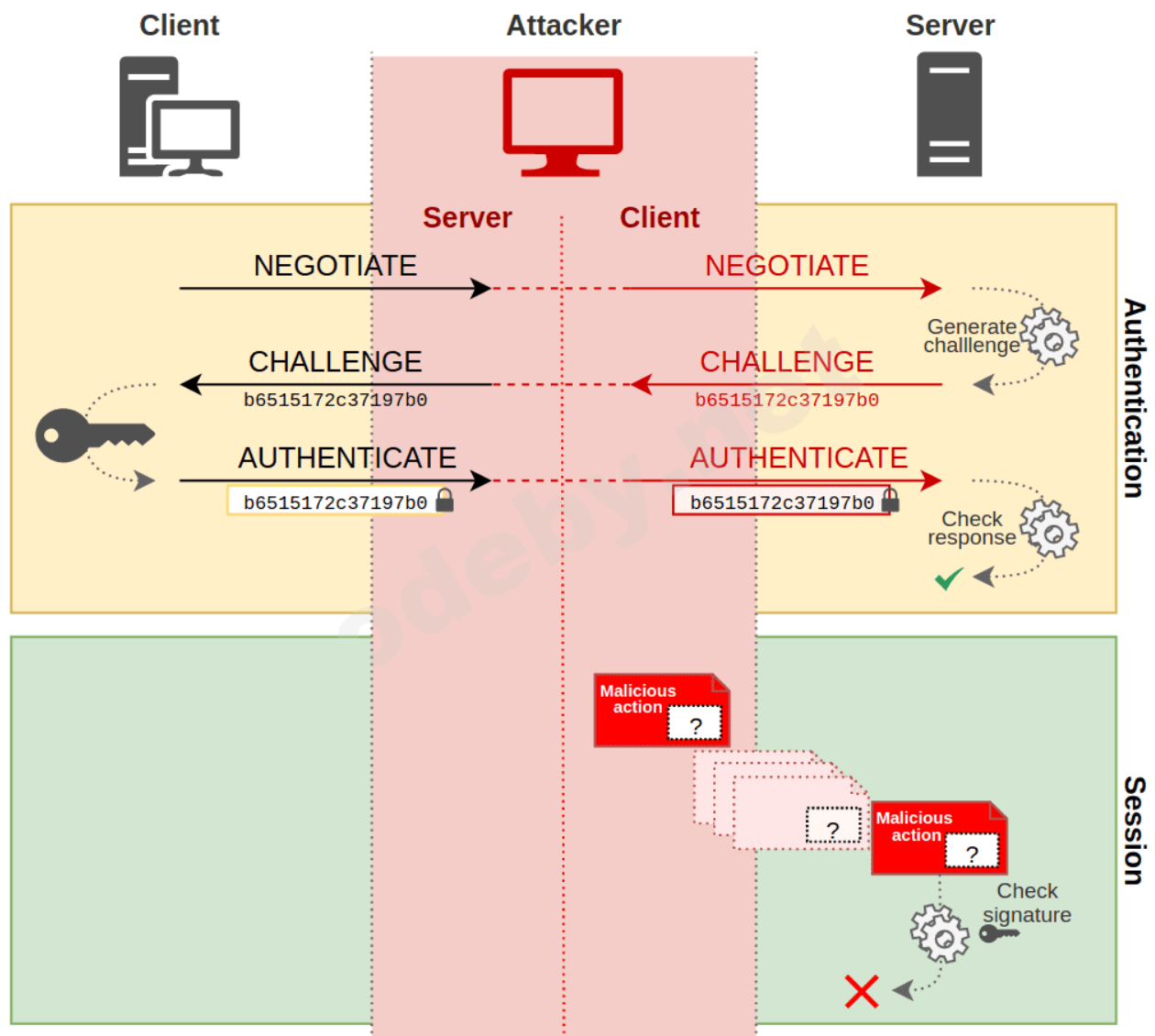
Принцип подписи может быть применен к любому обмену, если протокол его поддерживает. Это, например, касается [SMB](#), [LDAP](#) и даже [HTTP](#). Но на практике подпись HTTP сообщений реализуется редко.



Но тогда, какой смысл подписывать пакеты? Ну, как обсуждалось ранее, сеанс и аутентификация - это два отдельных шага, когда клиент хочет использовать службу. Поскольку злоумышленник может использовать man-in-the-middle атаку, и тем самым, находиться между клиентов и сервером и ретранслировать сообщения об аутентификации, он может и выдавать себя за клиента, когда связывается с сервером.

Именно здесь важную роль играет подпись. Даже если злоумышленнику удалось аутентифицироваться на сервере в качестве клиента, он не сможет подписывать пакеты, независимо от аутентификации. Действительно, чтобы иметь возможность подписывать пакет, необходимо **знать секрет** клиента.

Во время NTLM ретрансляции, злоумышленник хочет притвориться клиентом, но не знает его секрета. Поэтому он не может ничего подписать от его имени. Из-за того что он не может подписать ни один пакет, сервер, получающий пакеты, увидит, что подписи нет, и отклонит запрос злоумышленника.



Поэтому, вы можете понять, что если пакеты обязательно должны быть подписаны после аутентификации, то злоумышленник больше не сможет как-то действовать, так как он не знает секрета клиента. Значит, атака провалится. Данная мера защиты очень эффективна.

Это все очень хорошо, но как клиент и сервер договариваются о том, подписывать пакеты или нет? Это отличный вопрос.

Существует две вещи, которые вступают в игру.

- Первое - указать, **поддерживается ли подпись**. Это узнается во время NTLM Negotiation, или NTLM Согласования.
- Второе, это указать, подпись **required** (или обязательна), **optional** (необязательна) или **disabled** (отключенна). Это настройка, которая выполняется на клиентском и серверном уровне.

NTLM Negotiation, NTLM Согласование

Это согласование позволяет узнать, поддерживает ли клиент и/или сервер подпись(среди прочих вещей), и происходит это во время NTLM exchange, NTLM обмена. Поэтому я соврал вам немного выше, по поводу аутентификация и сессия, на самом деле, они не являются полностью независимыми. (Кстати, я сказал, что поскольку они независимы, вы можете менять протокол при ретрансляции, но есть ограничения, которые мы рассмотрим в главе о MIC при NTLM-аутентификации).

На самом деле, в сообщениях NTLM есть и другая информация, кроме вызова и ответа, которыми обмениваются клиент и сервер. Есть также negotiation flags, или **Negotiate Flags**. Эти флаги указывают, что поддерживает посылающая сторона.

Существует несколько флагов, но только один из них представляет интерес - **NEGOTIATE_SIGN**.

Когда клиентом установлен флаг на **1**, это означает, что клиент **поддерживает** подпись. Но будьте осторожны, это не значит, что он **обязательно подпишет** свои пакеты. Клиент подпишет только то, что он сможет подписать.

Аналогично, во время ответа сервера, если он поддерживает подпись, то флаг также будет установлен на **1**.

Таким образом, эти переговоры позволяют каждой из двух сторон, клиенту и серверу, указывать друг другу, способны ли они подписывать пакеты. Для некоторых протоколов, даже если клиент и сервер поддерживают подпись, то не обязательно означает, что они подписывают пакеты.

Реализация

Теперь, когда мы узнали, как обе стороны указывают друг другу на свою **способность** подписывать пакеты, они должны подписать их. На этот раз решение принимается согласно протоколу. Так что решение будет принято по-другому для SMBv1, для SMBv2, или для LDAP. Но идея остается прежней.

В зависимости от протокола, обычно существуют 2 или даже 3 опции, которые могут быть установлены, для решения, какой будет подпись: принудительной или нет. Этими 3 опциями являются:

- Disabled: Означает, что подпись не управляется.
- Enabled: Опция указывает на то, что машина может управлять подписанием при необходимости, но не требует подписи.
- Mandatory: Наконец, это указывает на: подпись поддерживается, пакеты **должны быть** подписаны, чтобы продолжилась сессия.

Здесь мы увидим пример двух протоколов, SMB и LDAP.

SMB

Матрица подписей

Матрица предоставляется в [документации Microsoft](#) для определения того, подписываются ли SMB-пакеты на основе настроек, на стороне клиента и на стороне сервера. Я описал это в таблице ниже. Обратите внимание, что для SMBv2 и выше, подписание обязательно обрабатывается, параметра **Disabled** больше нет.

SERVER CLIENT	Required	Enabled	Disabled (SMBv1)
	Required	Enabled	Disabled (SMBv1)
Required	Signed	Signed	Signed
Enabled	Signed*	SMBv1 : Signed SMBv2 : Not signed**	Not signed***
Disabled (SMBv1)	Signed	Not signed	Not signed

* Default for client/server to Domain Controller

** Default for client to server which is not a domain controller via SMBv2

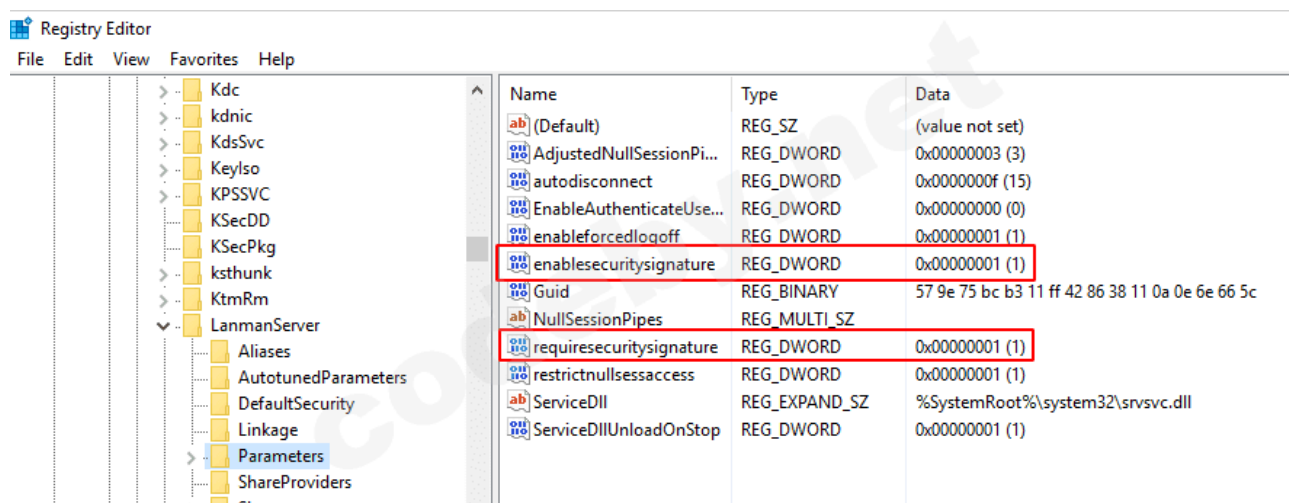
*** Default for client to server which is not a domain controller via SMBv1

Есть разница, когда клиент и сервер имеют параметр **Enabled(Включено)**. В SMBv1, параметром по умолчанию для серверов было **Disabled (Отключено)**. Таким образом, весь SMB-трафик между клиентами и серверами не был подписан по

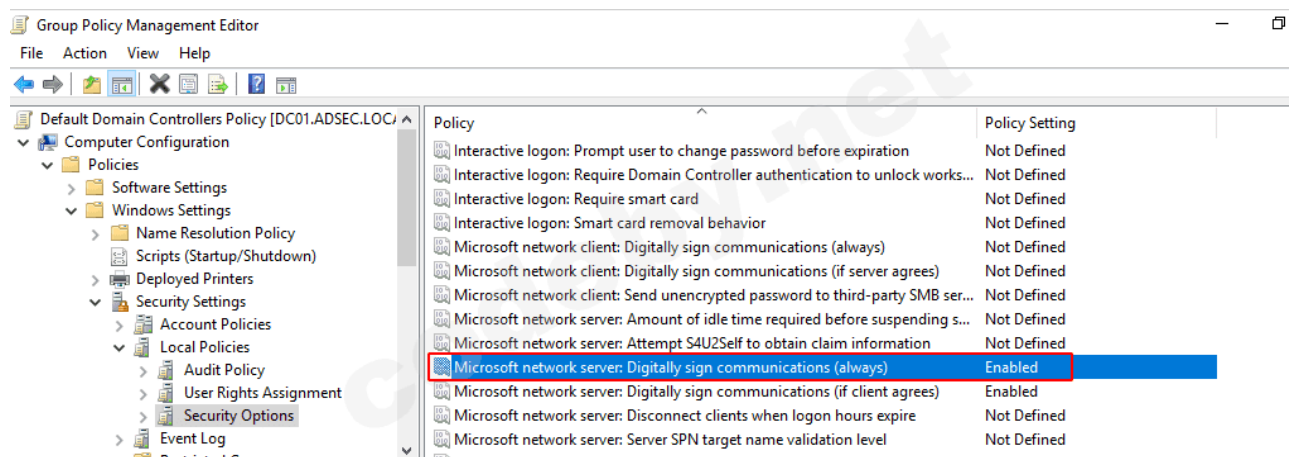
умолчанию. Это позволило избежать перегрузки серверов, не позволяя им вычислять подписи каждый раз при отправке SMB-пакета. Поскольку статуса **Disabled** больше не существует для SMBv2, и серверы теперь включены по умолчанию, то есть имеют опцию **Enabled**, для того, чтобы сохранить эту нагрузку, поведение между двумя включёнными объектами было изменено, и в этом случае подпись больше не требуется. Клиент и/или сервер должны обязательно **требовать** подпись для SMB пакетов.

Настройки

Чтобы изменить стандартные параметры подписи на сервере, ключи EnableSecuritySignature и RequireSecuritySignature должны быть изменены в разделе реестра
HKEY_LOCAL_MACHINE\System\CurrentControlSet\Services\LanmanServer\Parameters



Этот снимок экрана был сделан на контроллере домена. По умолчанию контроллеры домена требуют подписания SMB, когда клиент аутентифицируется на них. И это действительно так, GPO, примененный к контроллерам домена, содержит эту запись:



С другой стороны, на этом снимке мы видим, что над этой настройкой тот же самый параметр, применяемый к **сетевому клиенту Microsoft**, не применяется. Поэтому,

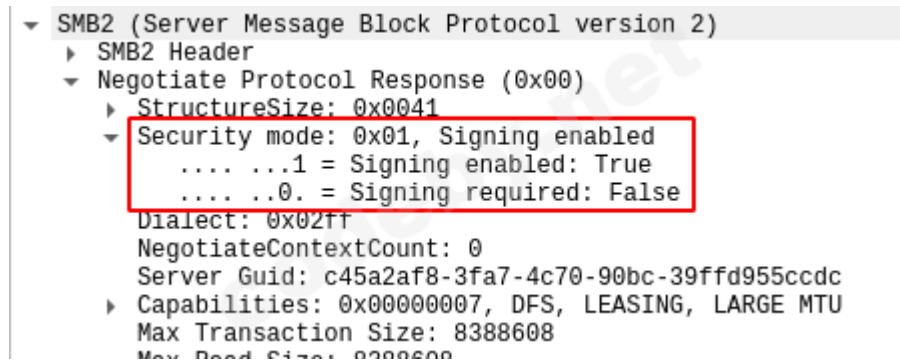
когда контроллер домена действует в качестве сервера SMB, требуется подпись SMB, но если соединение приходит с контроллера домена на сервер, подпись SMB не требуется.

Структура

Теперь, когда мы знаем, где настроена SMB подпись, мы видим, что этот параметр применяется во время взаимодействия. Это делается непосредственно перед аутентификацией. На самом деле, когда клиент подключается к SMB серверу, происходит следующее:

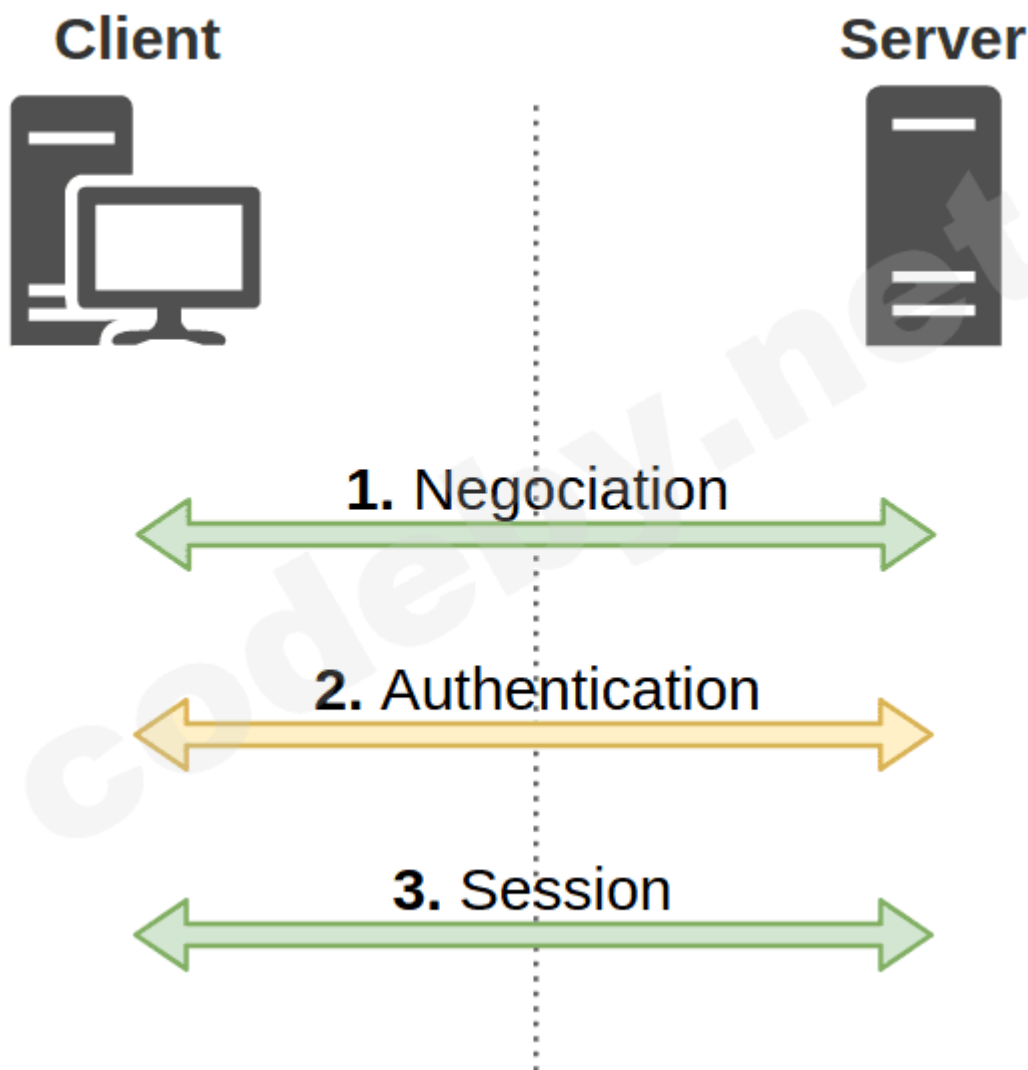
- Согласование по версии SMB и требования к подписи
- Аутентификация
- SMB сессия с согласованными параметрами

Вот пример согласования по SMB подписи:



Мы видим ответ сервера, указывающий, что он имеет параметр **Enable**, но не требует подписи.

Подводя итог, мы рассмотрим, как проходит согласование / аутентификация / сеанс :



1. На этапе переговоров обе стороны указывают свои требования: Требуется ли подпись для одного из них?
2. На этапе аутентификации обе стороны указывают, что они поддерживают. **Способны** ли они подписать?
3. На этапе сессии, если **возможности** и **требования** совместимы, сессия проводится с применением того, что было согласовано.

Например, если клиент **DESKTOP01** хочет взаимодействовать с контроллером домена **DC01**, **DESKTOP01** указывает, что он не требует подписи, но может справиться с этим, если это необходимо.

No.	Time	Source	Destination	Protocol	Length	Info
29	4.116765	192.168.56.211	192.168.56.201	SMB2	232	Negotiate Protocol Request
30	4.117174	192.168.56.201	192.168.56.211	SMB2	366	Negotiate Protocol Response

SMB2 (Server Message Block Protocol version 2)

SMB2 Header

Negotiate Protocol Request (0x00)

[Preauth Hash: c92ddf39b1f267de28ce41d2e0c0afce7c980741548d2276...]

StructureSize: 0x0024

Dialect count: 5

Security mode: 0x01, Signing enabled

.... ...1 = Signing enabled: True

.... ..0 = Signing required: False

Reserved: 0000

Capabilities: 0x0000007f, DFS, LEASING, LARGE MTU, MULTI CHANNEL, PERSISTENT HANDLES, DIRECTORY LEASING, ENCRYPTION

Client Guid: 6b07f029-6dc4-11ea-a942-080027f093cf

NegotiateContextOffset: 0x0070

NegotiateContextCount: 2

Reserved: 0000

Dialect: SMB 2.0.2 (0x0202)

DC01 указывает, что он не только поддерживает подпись, но и требует этого.

29	4.116765	192.168.56.211	192.168.56.201	SMB2	232	Negotiate Protocol Request
30	4.117174	192.168.56.201	192.168.56.211	SMB2	366	Negotiate Protocol Response

SMB2 (Server Message Block Protocol version 2)

SMB2 Header

Negotiate Protocol Response (0x00)

[Preauth Hash: 0aacc31a3b2402a2c65f4b68953fd93e87a5104b66484bd4...]

StructureSize: 0x0041

Security mode: 0x03, Signing enabled, Signing required

.... ...1 = Signing enabled: True

.... ...1 = Signing required: True

Dialect: SMB 3.1.1 (0x0311)

NegotiateContextCount: 2

Server Guid: bc759e57-11b3-42ff-8638-110a0e6e665c

Capabilities: 0x0000002f, DFS, LEASING, LARGE MTU, MULTI CHANNEL, DIRECTORY LEASING

Max Transaction Size: 8388608

Max Read Size: 8388608

Max Write Size: 8388608

Current Time: Mar 24, 2020 15:00:58.388259200 Romance Standard Time

На этапе согласования, клиент и сервер устанавливают флаг NEGOTIATE_SIGN равным 1, так как оба поддерживают подпись.

После завершения этой аутентификации сессия продолжается, и биржи малого и среднего бизнеса эффективно подписываются.

55	4.121457	192.168.56.211	192.168.56.201	SMB2	178 Ioctl Request FSCTL_QUERY_NETWORK_INTERFACE_INFO
56	4.121550	192.168.56.211	192.168.56.201	SMB2	190 Create Request File: wkssvc


```

> Transmission Control Protocol, Src Port: 49769, Dst Port: 445, Seq: 3366, Ack: 657, Len: 136
> NetBIOS Session Service
> SMB2 (Server Message Block Protocol version 2)
  > SMB2 Header
    ProtocolId: 0xfe534d42
    Header Length: 64
    Credit Charge: 1
    Channel Sequence: 0
    Reserved: 0000
    Command: Create (5)
    Credits requested: 1
  > Flags: 0x00000038, Signing, Priority
    ...0 = Response: This is a REQUEST
    ...0 = Async command: This is a SYNC command
    ...0 = Chained: This pdu is NOT a chained command
    ...1 = Signing: This pdu is SIGNED
    ...011 = Priority: This pdu contains a PRIORITY
    ...0 = DFS operation: This is a normal operation
    ...0 = Replay operation: This is NOT a replay operation
  Chain Offset: 0x00000000
  Message ID: Unknown (4)
  Process Id: 0x0000feff
  > Tree Id: 0x00000001 \\dc01\IPC$
  > Session Id: 0x0000400024000045
  Signature: e6e6a4a20bdbcd7201589bd4e915a3
  [Response in: 59]

```

LDAP

Матрица подписи

Для LDAP также существуют три уровня:

- Disabled: Это означает, что подписание пакетов не поддерживается.
- Negotiated Signing(Согласованная подпись): Эта опция указывает на то, что машина может обрабатывать подпись, и если машина, с которой она взаимодействует, также обрабатывает его, то они будут подписаны.
- Required: Это, наконец, указывает на то, что подписание поддерживается, и пакеты должны быть подписаны, чтобы сессия продолжилась.

Как вы можете прочесть, промежуточный уровень, **Negotiated Signing** отличается от случая SMBv2, потому что на этот раз, если клиент и сервер **смогут** подписать пакеты, то они **будут** их подписывать. В то время как для SMBv2 пакеты подписывались **только** в том случае, если это было требование по крайней мере для одного объекта.

Поэтому для LDAP у нас есть матрица, аналогичная SMBv1, за исключением поведения по умолчанию.

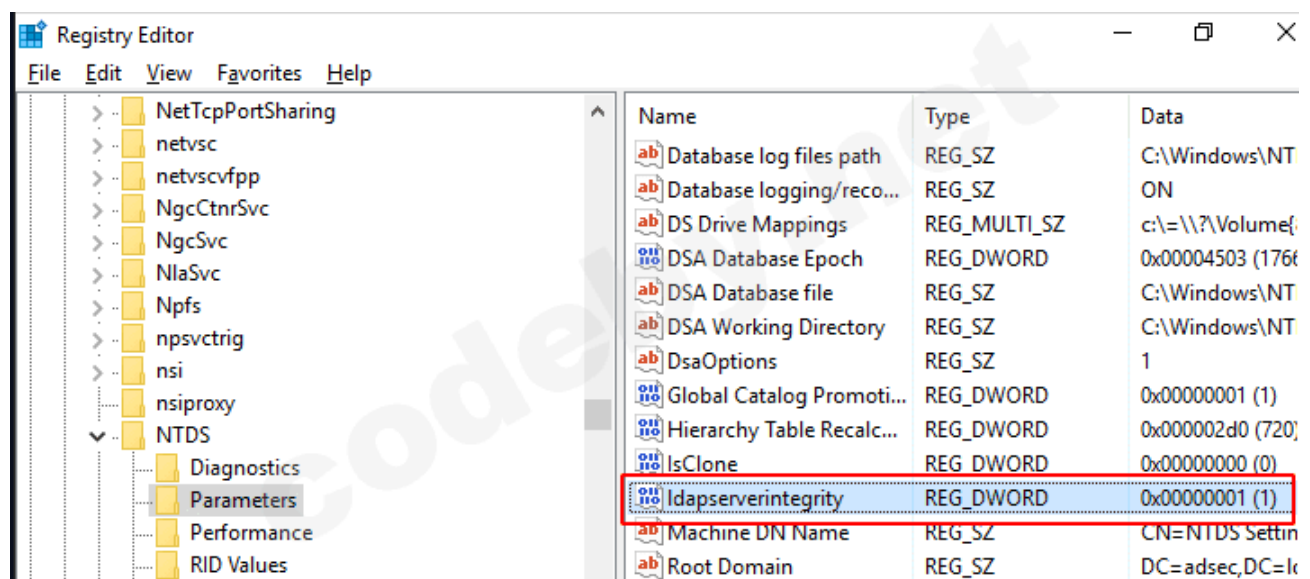
SERVER \ CLIENT	Required	Negotiated	Disabled
Required	Signed	Signed	Signed
Negotiated	Signed	Signed	Not signed
Disabled	Signed	Not signed	Not signed

* Default behavior

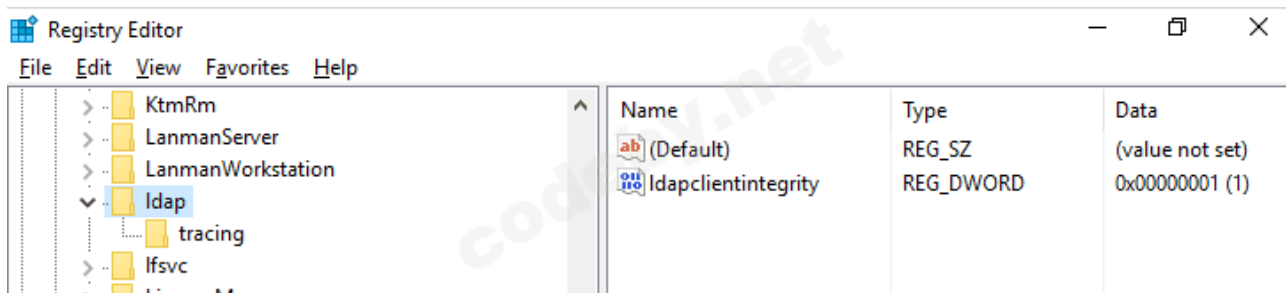
Разница с SMB заключается в том, что в домене Active Directory **все** узлы имеют настройку **Negotiated Signing**. Контроллер домена **не требует** подписи.

Настройки

Для **контроллера домена**, ключ реестра ldapserverintegrity находится в HKEY_LOCAL_MACHINE\System\CurrentControlSet\Services\NTDS\Parameters и может быть 0, 1 или 2 в зависимости от уровня. По умолчанию на контроллере домена установлено значение **1**.



Для **клиентов** этот ключ реестра находится в HKEY_LOCAL_MACHINE\System\CurrentControlSet\Services\ldap.



Для клиентов также установлено значение **1**. Поскольку все клиенты и контроллеры домена имеют **Negotiated Signing**, все LDAP пакеты подписываются по умолчанию.

Структура

В отличие от SMB, в LDAP нет флага, который бы указывал, будут ли пакеты подписаны или нет. Вместо этого LDAP использует флаги, установленные в NTLM согласованиях. Нет необходимости в дополнительной информации. В случае если и клиент, и сервер поддерживают подписание LDAP, то будет установлен флаг NEGOTIATE_SIGN и пакеты будут подписаны.

Если одна сторона **требует** подписи, а другая **не поддерживает**, то сессия просто не начнется. Сторона, требующая подписания, проигнорирует неподписанные пакеты.

Итак, теперь мы понимаем, что, в отличие от SMB, если мы находимся между клиентом и сервером и хотим передать аутентификацию на сервер, используя LDAP, нам нужны две вещи:

- **Сервер** не должен требовать подписи пакетов, как это происходит для всех машин по умолчанию
- **Клиент** не должен устанавливать флаг NEGOTIATE_SIGN на **1**, если он это сделает, то сервер будет ожидать подпись, а так как мы не знаем секрета клиента, мы не сможем подписывать наши созданные LDAP-пакеты.

Что касается требования **2**, то иногда клиенты не устанавливают этот флаг, но, к сожалению, клиент

Windows SMB делает это! По умолчанию невозможно передать SMB-аутентификацию в LDAP.

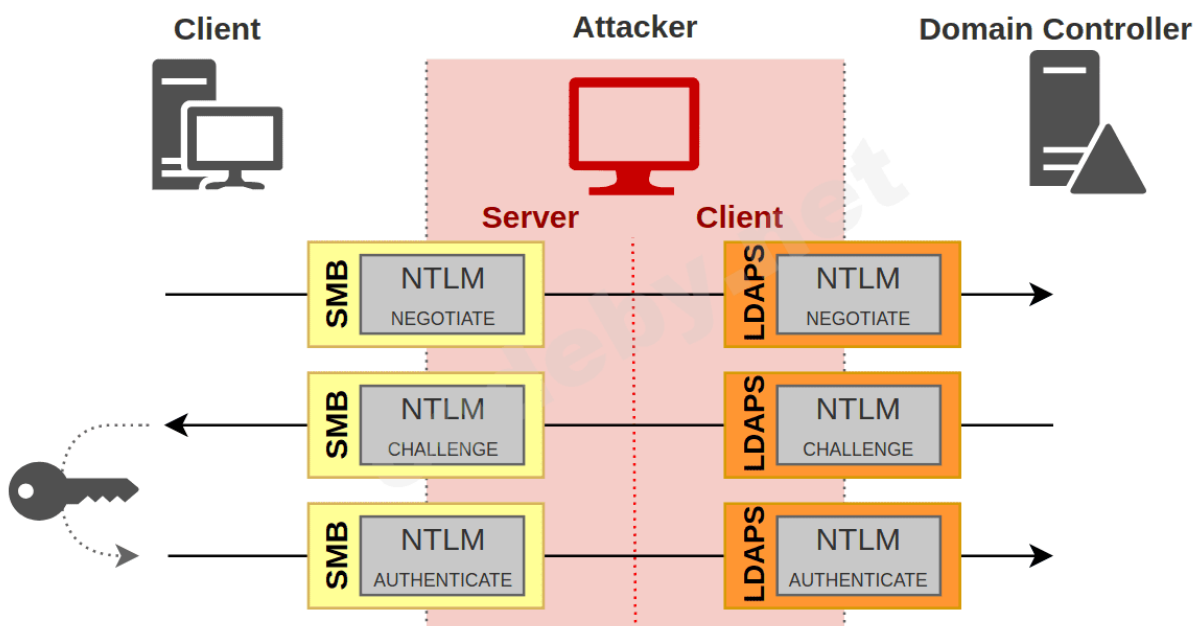
Так почему бы просто не поменять флаг NEGOTIATE_FLAG и не установить его на **0**? Ну... NTLM сообщения также подписаны. И это мы рассмотрим в следующем параграфе.

Подписание аутентификации (MIC)

Мы видели, как сеанс можно защитить от mitm атаки. Теперь, чтобы понять интерес этой главы, давайте рассмотрим конкретный случай.

Edge case

Давайте представим себе, что злоумышленник умудряется попасть в положение man-in-the-middle, между клиентом и контроллером домена, и что он получает запрос на аутентификацию через SMB. Зная, что контроллер домена требует SMB подписи, злоумышленник не может передать эту аутентификацию через SMB. С другой стороны, можно изменить протокол, как мы видели выше, и злоумышленник решает перейти на протокол **LDAPS**, так как аутентификация и сеанс должны быть независимыми.



Ну, они **почти** независимы.

Почти потому, что мы видели, что в данных аутентификации был флаг **NEGOTIATE_SIGN**, который присутствовал, что бы указать, поддерживают ли клиент и сервер подпись. Но в некоторых случаях этот флаг учитывается, как мы видели в LDAP.

Для LDAPS этот флаг также учитывается сервером. Если сервер получает запрос на аутентификацию с флагом **NEGOTIATE_SIGN**, установленным на **1**, он отклонит аутентификацию. Это происходит потому, что LDAPS является LDAP по TLS, и именно уровень TLS обрабатывает подписание (и шифрование) пакетов. Таким образом, у клиента LDAPS нет причин указывать, что он может подписывать свои пакеты, и если он утверждает, что может это сделать, сервер, в переносном значении, смеется над ним и хлопает дверью.

Теперь в нашей атаке клиент, которого мы ретранслируем, хочет аутентифицироваться через SMB, поэтому да, он поддерживает подписание пакетов, и да, установлен флаг **NEGOTIATE_SIGN** на **1**. Но если мы ретранслируем его

аутентификацию, ничего не меняя, через LDAPS, то LDAPS сервер увидит этот флаг, и завершит фазу аутентификации, без вопросов.

Мы могли бы просто изменить сообщение NTLM и удалить флаг, правильно? Если бы мы могли, то мы бы это сделали, это бы хорошо сработало. За исключением того, что на уровне NTLM также имеется подпись.

Эта подпись, она называется **MIC**, или **Message Integrity Code** (Код Целостности Сообщения).

MIC - Message Integrity Code

MIC - это подпись, которую отправляют только в последнем сообщении NTLM-аутентификации, в сообщении **AUTHENTICATE**. При этом учитываются другие 3 сообщения. MIC вычисляется функцией **HMAC_MD5**, используя в качестве ключа, который зависит от секрета клиента, называемого **session key**, **ключом сеанса**.

```
HMAC_MD5(Session key, NEGOTIATE_MESSAGE + CHALLENGE_MESSAGE +  
AUTHENTICATE MESSAGE)
```

Важно то, что ключ сеанса **зависит от секрета клиента**, поэтому злоумышленник не может создать MIC.

Вот пример MIC:

33	0.176816135	192.168.56.1	192.168.56.1	SMB	644 Session Setup AndX Request, NTLMSPP AUTH, User: ADSEC\jsnow
34	0.178676973	192.168.56.1	192.168.56.211	SMB2	681 Session Setup Request, NTLMSSP AUTH, User: ADSEC\jsnow
35	0.181379637	192.168.56.211	192.168.56.1	SMB2	171 Session Setup Response
36	0.182389041	192.168.56.1	192.168.56.211	SMB2	184 Tree Connect Request Tree: \\192.168.56.211\IPC\$

▼ SMB2 (Server Message Block Protocol version 2)

- ▶ SMB2 Header
- ▼ Session Setup Request (0x01)
 - ▶ StructureSize: 0x0019
 - ▶ Flags: 0
 - ▼ Security mode: 0x00
 -0 = Signing enabled: False
 -0 = Signing required: False
 - ▶ Capabilities: 0x00000000
 - Channel: None (0x00000000)
 - Previous Session Id: 0x0000000000000000
 - Blob Offset: 0x00000058
 - Blob Length: 523
 - ▼ Security Blob: a1820207308202030a0030a0101a28201e6048201e24e544c...
 - ▼ GSS-API Generic Security Service Application Program Interface
 - ▼ Simple Protected Negotiation
 - ▼ negTokenInArg
 - negResult: accept-incomplete (1)
 - responseToken: 4e544cd535350000300000180018007e0000003c013c01...
 - ▼ NTLM Secure Service Provider
 - NTLMSPP identifier: NTLMSSP
 - NTLM Message Type: NTLMSSP AUTH (0x00000003)
 - ▶ Lan Manager Response: 00
 - LMv2 Client Challenge: 0000000000000000
 - ▶ NTLM Response: f91306c7458a5d8b2863aaa2479e4bcf5101000000000000...
 - ▶ Domain name: ADSEC
 - ▶ User name: jsnow
 - ▶ Host name: DESKTOP01
 - ▶ Session Key: f17f73bf99088e88fe74cbb1e18efa85
 - ▶ Negotiate Flags: 0xe2888215, Negotiate 56, Negotiate Key Exchange, Negotiate 128, Negotiate Version, Negotiate Target I
 - ▶ Version 10.0 (Build 16299): NTLM Current Revision 15
 - MIC: 61cabfe042823f7208e8b3ed3483cb54

Таким образом, если только одно из 3 сообщений было изменено, то MIC больше не будет действительным, из-за несхожести конкатенации 3 сообщения. Поэтому вы не можете изменить флаг `NEGOTIATE_SIGN`, как это предлагается в нашем примере.

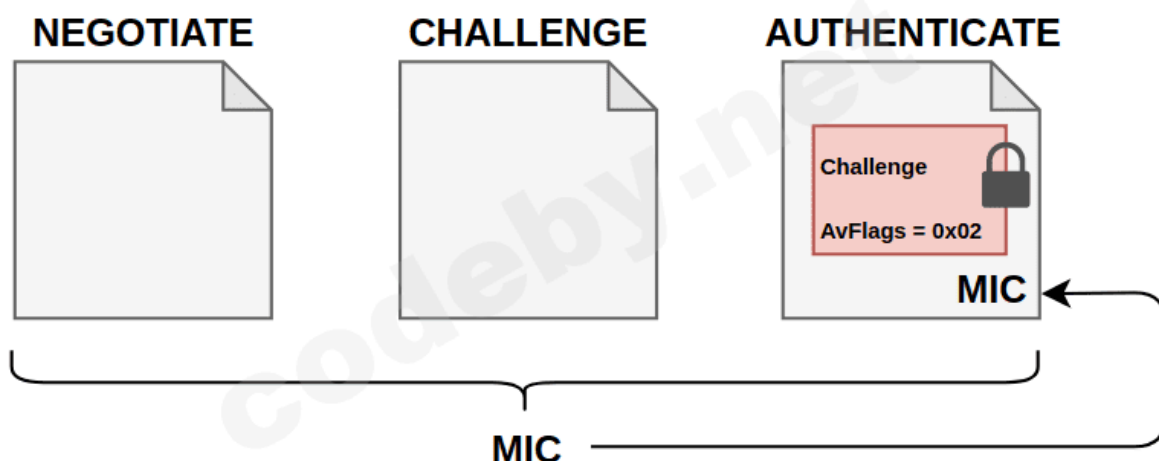
А что если мы просто удалим MIC? Потому что да, MIC необязателен.

[illegible]

...

14/31

Изменение или удаление этого флага делает хэш NTLMv2 недействительным, так как данные будут изменены. Вот как это выглядит.



MIC защищает целостность 3 сообщений, msAvFlags защищает присутствие MIC, а NTLMv2 хэш защищает присутствие флага. Атакующий, не зная секрета пользователя, не сможет подсчитать этот хэш.

Так что, вы должны были понять, что в этом случае мы ничего не можем сделать, и это благодаря MIC.

Drop the MIC

Небольшой обзор недавней уязвимости, найденной [Preempt](#), который вы легко сейчас поймете.

Это [CVE-2019-1040](#), красиво названный "Drop the MIC". Эта уязвимость показала, что если MIC был просто удален, даже если флаг указывал на его присутствие, то сервер принимал аутентификацию без проблем. Очевидно, это была ошибка, которую, с тех пор, пофиксили.

MIC был интегрирован в инструмент [ntlmrelayx](#) с помощью параметра `--remove-mic`.

Давайте рассмотрим наш пример ранее, но на этот раз с контроллером домена, который все еще уязвим. Вот как это выглядит на практике.


```

--$ sudo ntlmrelayx.py -t ldaps://192.168.56.201 --remove-mic -smb2support
Impacket v0.9.21-dev - Copyright 2019 SecureAuth Corporation

[*] Protocol Client HTTP loaded..
[*] Protocol Client HTTPS loaded..
[*] Protocol Client IMAP loaded..
[*] Protocol Client IMAPS loaded..
[*] Protocol Client LDAPS loaded..
[*] Protocol Client LDAP loaded..
[*] Protocol Client MSSQL loaded..
[*] Protocol Client SMB loaded..
[*] Protocol Client SMTP loaded..
[*] Running in relay mode to single host
[*] Setting up SMB Server
[*] Setting up HTTP Server

[*] Servers started, waiting for connections
[*] SMBD-Thread-3: Received connection from 192.168.56.211, attacking target ldaps://192.168.56.201
[*] Authenticating against ldaps://192.168.56.201 as ADSEC\jsnow SUCCEED
[*] Enumerating relayed user's privileges. This may take a while on large domains
[*] SMBD-Thread-5: Received connection from 192.168.56.211, attacking target ldaps://192.168.56.201
[*] Authenticating against ldaps://192.168.56.201 as ADSEC\jsnow SUCCEED
[*] Enumerating relayed user's privileges. This may take a while on large domains
[*] SMBD-Thread-7: Received connection from 192.168.56.211, attacking target ldaps://192.168.56.201
[*] Authenticating against ldaps://192.168.56.201 as ADSEC\jsnow SUCCEED
[*] Enumerating relayed user's privileges. This may take a while on large domains

```

Наша атака работает. Отлично.

Для информации, еще одна уязвимость была обнаружена той же самой командой, и они назвали ее [Drop The MIC 2](#).

Ключ сеанса

Ранее мы говорили о подписи сеанса и аутентификации, говоря, что для того, чтобы подписать что-то, нужно знать секрет пользователя. В главе о MIC мы сказали, что на самом деле используется не секрет пользователя, а ключ, называемый **ключом сеанса**, который зависит от секрета пользователя.

Чтобы дать вам представление, вот как вычисляется ключ сеанса для NTLMv1 и NTLMv2.

Код:

```

# For NTLMv1
Key = MD4(Hash NT)

# For NTLMv2
Key = HMAC_MD5(NTLMv2 Hash, HMAC_MD5(NTLMv2 Hash, NTLMv2 Response + Challenge))

```

Понять объяснения было бы не очень полезно, но очевидно, что в разных версиях есть разница в сложности. Повторяю, **не используйте NTLMv1 в производственной сети**.

Имея эту информацию, понимаем, что клиент может вычислить этот ключ на своей стороне, так как у него есть для этого вся информация.

Сервер, с другой стороны, не всегда может сделать это в одиночку. Для [локальной аутентификации](#) нет никаких проблем, так как сервер знает NT хэш пользователя.

С другой стороны, для [аутентификации с учетной записью домена](#) серверу придется попросить контроллер домена вычислить ключ сеанса для него и отправить его обратно. В pass-the-hash статье мы увидели, что сервер посылает запрос на контроллер домена в структуре [NETLOGON_NETWORK_INFO](#), а контроллер домена отвечает структурой [NETLOGON_VALIDATION_SAM_INFO4](#). Именно в этом ответе с контроллера домена отправляется ключ сеанса, если аутентификация прошла успешно.

```
typedef struct _NETLOGON_VALIDATION_SAM_INFO4 {
    OLD_LARGE_INTEGER LogonTime;
    OLD_LARGE_INTEGER LogoffTime;
    OLD_LARGE_INTEGER KickOffTime;
    OLD_LARGE_INTEGER PasswordLastSet;
    OLD_LARGE_INTEGER PasswordCanChange;
    OLD_LARGE_INTEGER PasswordMustChange;
    RPC_UNICODE_STRING EffectiveName;
    RPC_UNICODE_STRING FullName;
    RPC_UNICODE_STRING LogonScript;
    RPC_UNICODE_STRING ProfilePath;
    RPC_UNICODE_STRING HomeDirectory;
    RPC_UNICODE_STRING HomeDirectoryDrive;
    unsigned short LogonCount;
    unsigned short BadPasswordCount;
    unsigned long UserId;
    unsigned long PrimaryGroupId;
    unsigned long GroupCount;
    [size_is(GroupCount)] PGROUP_MEMBERSHIP GroupIds;
    unsigned long UserFlags;
    USER_SESSION_KEY UserSessionKey;
    RPC_UNICODE_STRING LogonServer;
    RPC_UNICODE_STRING LogonDomainName;
    PRPC_SID LogonDomainId;
    unsigned char LMKey[8];
    ULONG UserAccountControl;
    ULONG SubAuthStatus;
    OLD_LARGE_INTEGER LastSuccessfulILogon;
    OLD_LARGE_INTEGER LastFailedILogon;
    ULONG FailedILogonCount;
    ULONG Reserved4[1];
    unsigned long SidCount;
```

Затем возникает вопрос о том, что мешает злоумышленнику сделать тот же запрос к контроллеру домена, что и к целевому серверу. Ну, до [CVE-2015-005](#), ничего!

Во время реализации протокола NETLOGON [12] мы обнаружили, что контроллер домена, не проверяющий, была ли отправляемая информация аутентификации, на самом деле предназначен для компьютера,

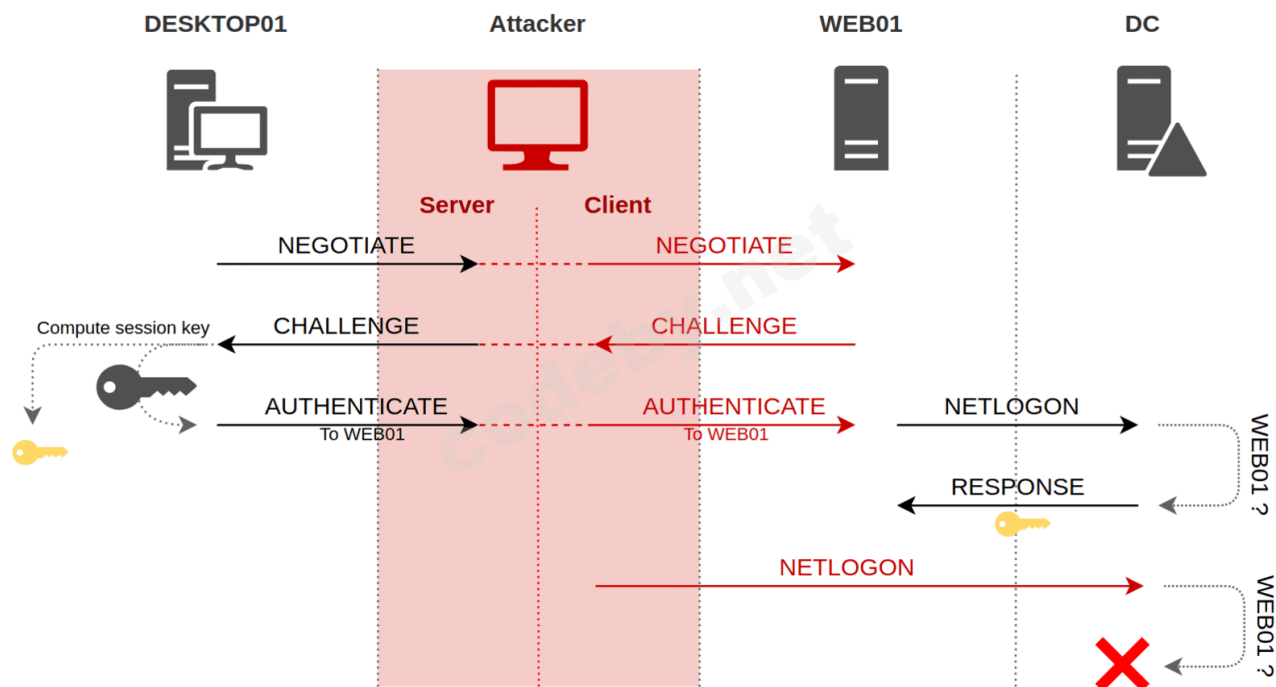
присоединенный к домену, который запрашивает эту операцию (например, `NetrLogonSamLogonWithFlags()`). Это означает, что **любой подключенный к домену компьютер может проверить любую проходную аутентификацию на контроллере домена и получить базовый ключ для криптографических операций для любой сессии в домене.**

Очевидно, Майкрософт исправил эту ошибку. Для проверки того, что только сервер, на котором пользователь аутентифицируется, имеет право запросить ключ сеанса, контроллер домена убедится, что целевой компьютер в ответе AUTHENTICATE тот же самый, что и хост, делающий NetLogon запрос.

В ответе AUTHENTICATE мы подробно описали присутствие `msAvFlags`, указывающих на присутствие или отсутствие MIC, но есть и другая информация, например, Netbios название целевого компьютера.

```
▼ NTLM Secure Service Provider
  NTLMSSP identifier: NTLMSSP
  NTLM Message Type: NTLMSSP_AUTH (0x00000003)
  ▶ Lan Manager Response: 0000000000000000000000000000000000000000000000000000000000000000
  LMv2 Client Challenge: 0000000000000000
  ▼ NTLM Response: f91306c7458a5d8b2863aaa2479e4bcf0101000000000000...
    Length: 316
    Maxlen: 316
    Offset: 150
  ▼ NTLMv2 Response: f91306c7458a5d8b2863aaa2479e4bcf0101000000000000...
    NTProofStr: f91306c7458a5d8b2863aaa2479e4bcf
    Response Version: 1
    Hi Response Version: 1
    Z: 000000000000
    Time: Mar 24, 2020 11:18:02.156601100 UTC
    NTLMv2 Client Challenge: e39c15d3ffeeb1d7
    Z: 00000000
    ▶ Attribute: NetBIOS domain name: ADSEC
    ▶ Attribute: NetBIOS computer name: SERVER01
    ▶ Attribute: DNS domain name: adsec.local
    ▶ Attribute: DNS computer name: SERVER01.adsec.local
    ▶ Attribute: DNS tree name: adsec.local
    ▶ Attribute: Timestamp
    ▶ Attribute: Flags
    ▶ Attribute: Restrictions
    ▶ Attribute: Channel Bindings
    ▶ Attribute: Target Name: cifs/192.168.56.1
    ▶ Attribute: End of list
    Z: 00000000
    padding: 00000000
```

Это имя сравнивается с именем хоста, выполняющего запрос NetLogon. Таким образом, если злоумышленник попытается сделать NetLogon запрос на ключ сеанса, так как имя злоумышленника не совпадает с именем целевого узла в ответе NTLM, контроллер домена отклонит запрос.



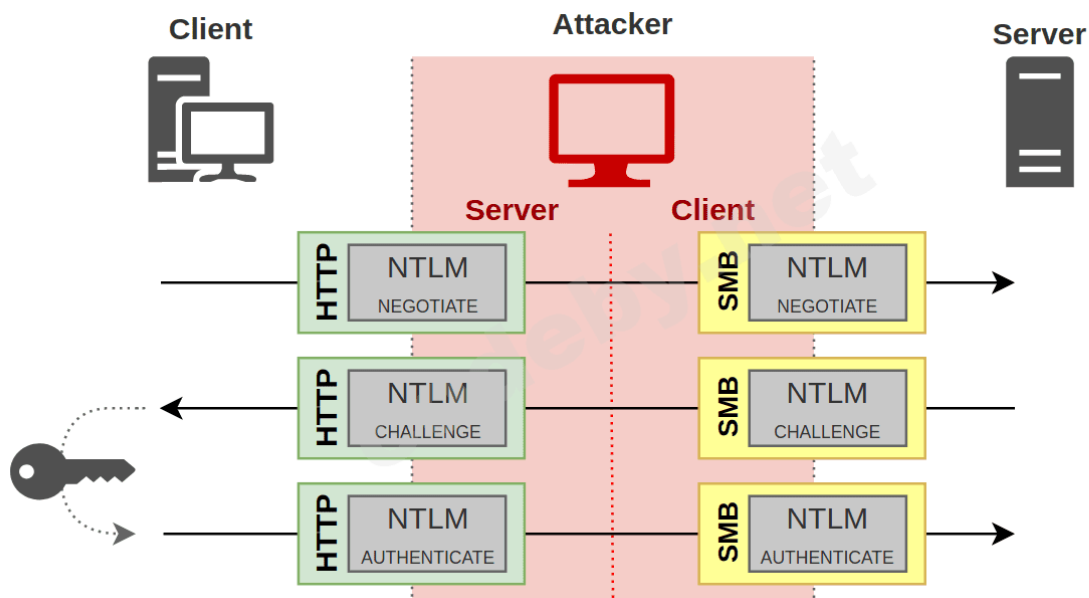
Наконец, как и в случае с msAvFlags, мы не можем изменить имя компьютера "на лету" в ответе NTLM, так как оно учитывается при расчете ответа NTLMv2.

Уязвимость, похожая на **Drop the MIC 2**, была недавно обнаружена группой безопасности Preempt. Вот [ссылка](#) на их пост, если вам интересно.

Channel Binding

Мы поговорим об одном последнем пункте. Несколько раз мы повторяли, что уровень аутентификации, т.е. NTLM сообщений, практически не зависит от прикладного уровня, используемого протокола (SMB, LDAP, ...). Я говорю "почти", потому что мы видели, что некоторые протоколы используют некие NTLM-флаги сообщений, чтобы узнать, должен ли сеанс быть подписан или нет.

В любом случае, как уже говорилось, злоумышленник вполне может получить NTLM-сообщение от протокола А и отправить его обратно по протоколу В. Это называется **cross-protocol relay**, **межпротокольным реле**, о чем мы уже говорили.



В целом, существует новая защита, противостоящая этой атаке. Это называется **channel binding**, или **EPA** (Enhanced Protection for Authentication). Принцип этой защиты заключается в «связывании» уровня аутентификации с используемым протоколом, даже с уровнем TLS, когда он существует (например, LDAPS или HTTPS). Общая идея заключается в том, что в последнем сообщении NTLM AUTHENTICATE помещается часть информации, которая не может быть изменена злоумышленником. Эта информация указывает на желаемую службу, а так же, возможно, содержит хэш сертификата целевого сервера.

Мы рассмотрим эти два принципа немного подробнее, но не волнуйтесь, все, в принципе, можно легко понять.

Service binding

Эту первую защиту легко понять. Если клиент желает аутентифицироваться на сервере для использования определенного сервиса, информация, идентифицирующая сервис, будет добавлена в ответ NTLM.

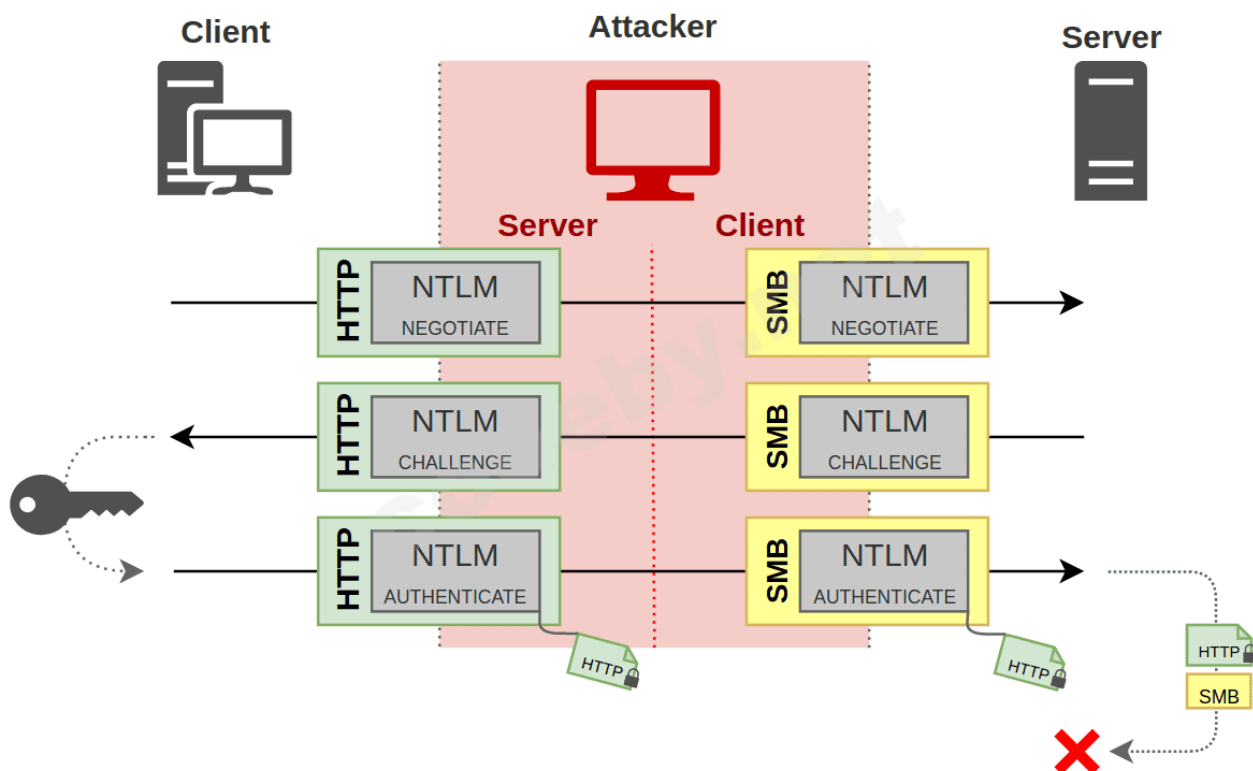
Таким образом, когда легитимный сервер получает эту аутентификацию, он может увидеть сервис, который был запрошен клиентом, и если он отличается от того, что было на самом деле запрошено, он не согласится предоставить сервис.

Так как имя сервиса находится в ответе NTLM, то оно защищено ответом **NtProofStr**, который является HMAC_MD5 этой информации, вызовом, и другой информацией, такой как **msAvFlags**. Это вычисляется с секретом клиента.

В примере, показанном на последней диаграмме, мы видим клиента, пытающегося аутентифицироваться по HTTP на сервере. За исключением того, что сервер является злоумышленником, и злоумышленник просто не может

аутентифицироваться на легитимном сервере, чтобы получить доступ к сетевому ресурсу (SMB).

Кроме того, что клиент указал в своем ответе NTLM сервис, который он хочет использовать, и поскольку атакующий не может его модифицировать, он должен передать его как есть. Затем сервер получает последнее сообщение, сравнивает службу, запрошенную атакующим (SMB) со службой, указанной в сообщении NTLM (HTTP), и отказывает в соединении, обнаруживая, что эти две службы не совпадают.



Конкретно, то, что называется **сервисом**, на самом деле является SPN или Service Principal Name (Имя Руководителя Службы). Я посвятил [целую статью](#) объяснению этого понятия.

Вот скриншот клиента, посылающего SPN в своем NTLM ответе.

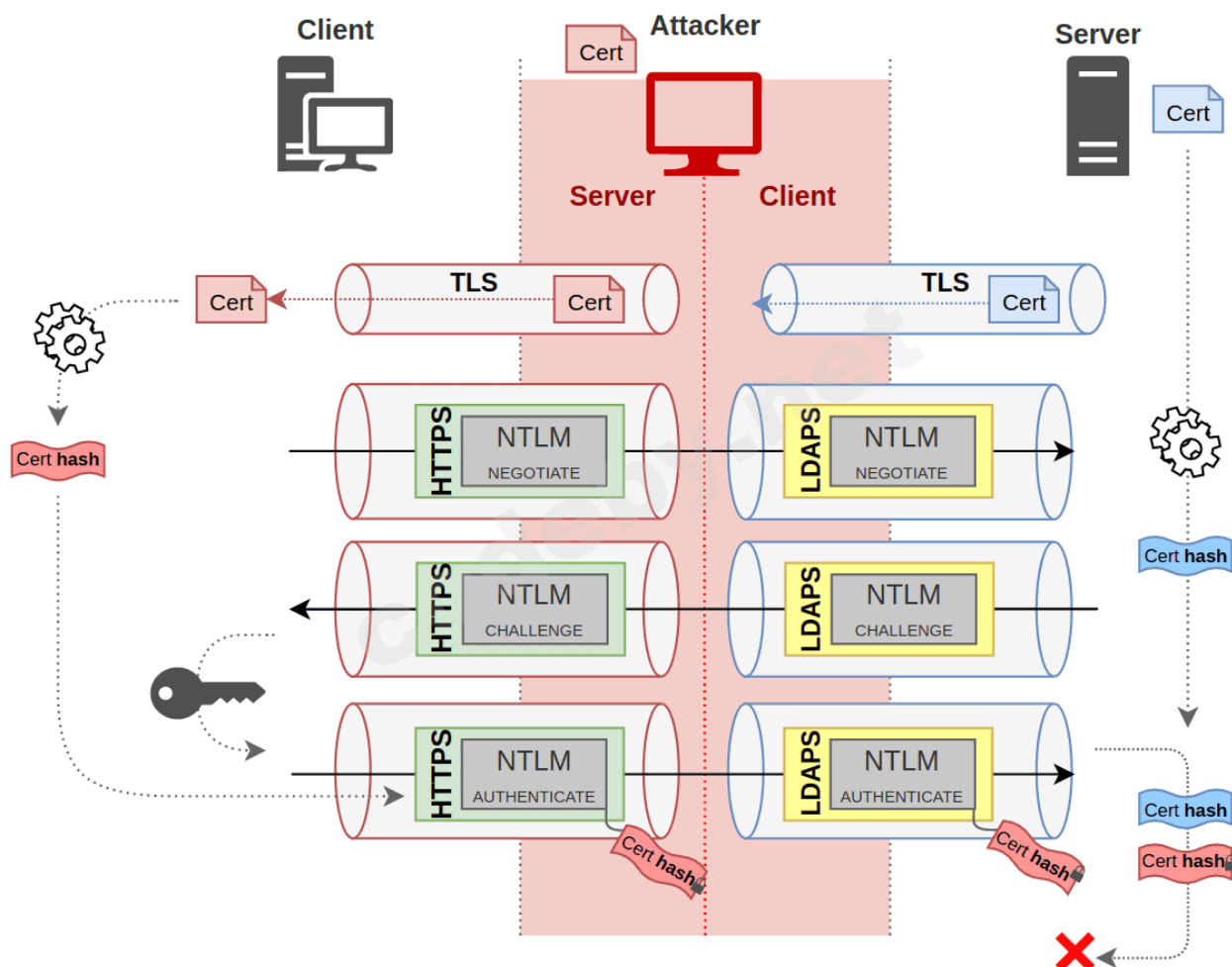
и сравнит его с реальным хэшем своего сертификата. Если он отличается, значит, он не был первоначальным получателем NTLM обмена.

Опять же, поскольку этот хэш находится в NTLM ответе, он защищен ответом **NtProofStr**, как и **SPN** для **Service Binding**.

Благодаря этой защите, следующие две атаки больше не возможны:

- Если атакующий хочет передать информацию от клиента, использующего протокол **без** уровня TLS, на протокол **с** уровнем TLS (например, HTTP на LDAPS), он не сможет добавить хэш сертификата с целевого сервера в NTLM ответ, поскольку не сможет обновить NtProofStr.
- Если атакующий хочет переслать протокол с TLS на другой протокол с TLS (HTTPS на LDAPS), то при создании сеанса TLS между клиентом и атакующим, атакующий не сможет предоставить сертификат сервера, так как он не совпадает с идентификацией атакующего. Поэтому он должен будет предоставить самодельный сертификат, идентифицирующий злоумышленника. Затем клиент будет хэшировать этот сертификат, и когда злоумышленник передаст NTLM ответ легитимному серверу, хэш в ответе не будет совпадать с хэшем реального сертификата, поэтому сервер отвергнет аутентификацию.

Вот диаграмма для иллюстрации второго случая. Кажется сложным, но это не так.



Это показывает создание двух сессий TLS. Один между клиентом и атакующим (красным) и один между атакующим и сервером (синим). Клиент получит сертификат злоумышленника и вычислит хэш, **сертификат хэша**, который обозначен красным цветом.

В конце NTLM обменов этот хэш будет добавлен в NTLM ответ и будет защищен, так как он является частью зашифрованных данных NTLM ответа. Когда сервер получит этот хэш, он будет хэшировать свой сертификат, и, увидев, что это не тот же результат, он откажется от соединения.

Опять же, Preempt недавно [обнаружил уязвимость](#), которая была исправлена с тех пор.

Что может быть передано?

Имея всю эту информацию, вы должны знать, какие протоколы можно передавать по каким протоколам. Мы видели, что невозможно осуществить ретрансляцию, например, с SMB на LDAP или LDAPS. С другой стороны, любой клиент, который не установил флаг NEGOTIATE_SIGN, может быть ретранслирован в LDAP.

Так как существует множество случаев, вот таблица, обобщающая некоторые из них.

SERVER CLIENT			Signing						Channel binding					
			Disabled		Enabled		Required		Disabled		Enabled		Required	
			SMB v1	HTTP	SMB v2	LDAP	SMB	LDAP	LDAPS	HTTPS	LDAPS	HTTPS	LDAPS	HTTPS
Signing	Disabled	SMB v1												
		HTTP												
	Enabled	SMBv2												
		LDAP												
	Required	SMB												
		LDAP												
Channel binding	Disabled	LDAPS												
		HTTPS												
	Enabled	LDAPS												
		HTTPS												
	Required	LDAPS												
		HTTPS												

Я думаю, что мы не можем ретранслировать LDAPS или HTTPS, поскольку у злоумышленника нет действительного сертификата, но, допустим, клиент разрешает использование недоверенных сертификатов. Могут быть добавлены и другие протоколы, такие как SQL или SMTP, но я должен признать, что не прочитал документацию по всем существующим протоколам.

Для gray box я не знаю, как HTTPS-сервер обрабатывает аутентификацию с флагом NEGOTIATE_SIGN, установленным на 1.

Перестаньте. Использовать. NTLMv1.

Вот небольшой "забавный факт", который [Марина Симаков](#) предложила мне добавить. Как мы обсуждали, NTLMv2 хэш учитывает вызов сервера, а также флаг msAvFlags, который указывает на наличие MIC поля, указывающего NetBios имя

целевого узла во время аутентификации, SPN в случае привязки сервиса, и хэш сертификата в случае привязки TLS.

В принципе, протокол NTLMv1 **этого не делает**. При этом учитывается только вызов сервера. Фактически, больше нет никакой дополнительной информации, такой как имя цели, msAvFlags, SPN или CBT.

Таким образом, если NTLMv1-аутентификация разрешена сервером, злоумышленник может просто удалить MIC и тем самым передать аутентификацию, например, LDAP или LDAPS.

Но что более важно, он может сделать NetLogon запросы на получение ключа сеанса. На самом деле, контроллер домена не имеет возможности проверить, имеет ли он на это право. А так как он не блокирует производственную сеть, которая не полностью обновлена, он любезно передаст ее злоумышленнику, по причине "ретро-совместимости".

Как только у него будет ключ сеанса, он может подписать любой пакет, который захочет. Это означает, что даже если цель попросит подписать, он сможет это сделать.

Так и должно быть, и оно не может быть исправлено. Так что повторяю, **не разрешайте NTLMv1 в производственной сети**.

Заключение

Итак, было много информации.

Мы видели здесь подробности NTLM ретрансляции, зная, что аутентификация и следующая за ней сессия являются двумя различными понятиями, позволяющими во многих случаях выполнять **межпротокольную ретрансляцию**. Хотя протокол каким-то образом включает в себя данные аутентификации, он непрозрачен для протокола и управляется SSPI.

Мы также показали, как **подписание пакетов** может защитить сервер от атак типа man-in-the-middle. Для этого цель должна ждать подписанного пакета, идущего от клиента, иначе злоумышленник сможет притвориться кем-то другим, и без необходимости подписывать посылаемые им сообщения.

Мы увидели, что MIC очень важен для защиты NTLM-обменов, особенно флаг, указывающий, будут ли пакеты подписаны для определенных протоколов, или информация о привязке каналов.

Мы остановились на том, как привязка каналов может «связать» уровень аутентификации и сеансовый уровень, либо через имя службы, либо через ссылку с сертификатом сервера.

Я надеюсь, что эта статья дала вам понимание того, что происходит во время атаки на NTLM ретранслятор и как от этого защититься.

Поскольку эта статья весьма существенна, вполне вероятно, что некоторые ошибки проскользнули внутрь. Не стесняйтесь связаться со мной в [twitter](#) или на моем [Discord Server](#), чтобы обсудить это.

Последнее редактирование:

Администратор

31.12.2015

6 326

7 014

Специализация

- [Добавить закладку](#)
- [#2](#)

Очень длинное предложение. Люди так не общаются. Надо разбить на 2-3 предложения.

[g00db0y](#)

Red Team

12.06.2018

137

687

- [Добавить закладку](#)
- [#3](#)

[The Codeby](#) сказал(а):

Очень длинное предложение. Люди так не общаются. Надо разбить на 2-3 предложения.

Исправил



[renat baidukov](#)

Green Team

17.06.2020

10

1

- [Добавить закладку](#)
- [#4](#)

Наверно это полезно знать. Но как-то очень уж много информации. Я писал SMB сканеры 1 и 2 версии причем сам собирал SMB пакеты, и встраивал в приложения NTLM авторизацию. Но никогда так глубоко не копался в этих дебрях. Когда будет очень скучно и мне захочется сделать SMB-proxu обязательно перечитаю все внимательно.

[Tak10](#)

Green Team

04.09.2025

13

3

- [Добавить закладку](#)
- [#5](#)

Хорошая статья, но некоторые моменты вызывают вопросы. 1)В начале первой части статьи приводится терминология и я никак не могу понять следующую формулировку: "NTLMv1 Hash и NTLMv2 Hash будут терминами, обозначающие ответ на запрос клиента, для 1 и 2 версий протокола NTLM". То есть термином NTLMv2 Hash в статье обозначается сообщение NTLM AUTHENTICATE содержащее хэшированный клиентом челлендж, или сообщение NTLM CHALLENGE собственно содержащее челлендж который клиент должен хэшировать? 2)Далее ниже по тексту, в главе "Net-NTLMv1 и Net-NTLMv2" есть следующая цитата: "Для информации, именно этот действительный ответ, переданный атакующим в 3 сообщении, является

зашифрованным вызовом, который обычно называется Net-NTLMv1 хэш или Net-NTLMv2 хэш. Но в этой статье она будет называться NTLMv1 хэшем или NTLMv2 хэшем, как указано в предварительном параграфе". Из этой цитаты я делаю вывод что термин NTLMv2 Hash все таки обозначает именно часть сообщения NTLM AUTHENTICATE содержащую хэшированную версию челленджа? 3) Во второй части статьи, в главе про MIC есть следующая цитата: "NTLMv2 Hash, который является ответом на вызов, посланный сервером, представляет собой хэш, который учитывает не только вызов (очевидно), но и все флаги ответа". Из этой цитаты я делаю вывод что термин NTLMv2 Hash обозначает хэшированные ключом клиента челлендж и все флаги сообщения NTLM AUTHENTICATE. Но тут возникает расхождение с оригиналом статьи в которой указан следующий алгоритм расчета сессионного ключа: # For NTLMv2 NTLMv2 Hash = HMAC_MD5(NT Hash, Uppercase(Username) + UserDomain) Key = HMAC_MD5(NTLMv2 Hash, HMAC_MD5(NTLMv2 Hash, NTLMv2 Response + Challenge)) Отсюда можно сделать вывод что термин NTLMv2 Hash это значение полученное в результате применения HMAC_MD5 к имени учетной записи и имени домена с использованием в качестве ключа nt хэша учетной записи. Отсюда возникает путаница с алгоритмом расчета сессионного ключа, а точнее с тем что значит термин NTLMv2 Hash в приведенном алгоритме расчета сессионного ключа. Может ктонибудь, пожалуйста, наглядно объяснить что в итоге значит термин NTLMv2 Hash в контексте алгоритма расчета сессионного ключа?

Последнее редактирование:

31.12.2015

6 326

7 014

Специализация

- [Добавить закладку](#)
- [#6](#)

Здравствуйте! Благодарю за ваш вдумчивый комментарий и отличные вопросы. Вы совершенно правы, терминология вокруг NTLM может быть запутанной, и в статье она используется не всегда однозначно. Давайте разберем ваши пункты, чтобы внести ясность.

Вопросы 1 и 2: Что такое NTLMv2 Hash - ответ клиента или вызов сервера?

Вы абсолютно верно пришли к выводу во втором пункте. В контексте статьи, когда говорится об "NTLMv1/v2 Hash" или "Net-NTLMv1/v2 Hash", имеется в виду именно ответ клиента на вызов (challenge) сервера.

- NTLM CHALLENGE (сообщение 2): Сервер отправляет клиенту случайную строку — "вызов".
- NTLM AUTHENTICATE (сообщение 3): Клиент, зная пароль пользователя, обрабатывает этот "вызов" и отправляет обратно "ответ".

Именно этот ответ, содержащийся в сообщении NTLM AUTHENTICATE, в статье и называется "NTLMv2 Hash". Это доказательство того, что клиент владеет паролем, не передавая сам пароль по сети.

Вопрос 3: Путаница в алгоритме расчета сессионного ключа

Это самый важный и точный ваш вопрос, который вскрывает ключевую неоднозначность. Вы столкнулись с ситуацией, где один и тот же термин — "NTLMv2 Hash" — используется для обозначения двух разных сущностей. Это частая причина путаницы. Давайте разложим всё по полочкам.

Шаг 1: Промежуточный хэш (тот, что из формулы)

Сначала вычисляется хэш, который является производным от пароля пользователя, его имени и домена. Назовем его промежуточным ключом. Именно он фигурирует в первой строке вашего примера:

$$\text{NTLMv2 Hash} = \text{HMAC_MD5}(\text{NT Hash}, \text{Uppercase}(\text{Username}) + \text{UserDomain})$$

Здесь:

- NT Hash — это хэш пароля пользователя (стандартный NTLM-хэш).
- Username и UserDomain — имя пользователя и домен.

Этот NTLMv2 Hash НЕ является ответом на вызов сервера. Это, по сути, усиленный ключ, специфичный для пользователя, который будет использоваться на следующих шагах.

Шаг 2: Ответ клиента (тот, что летит по сети)

Теперь, используя промежуточный ключ из Шага 1, клиент формирует тот самый ответ на вызов сервера (NTLMv2 Response), который отправляется в сообщении NTLM AUTHENTICATE. Его формула (в упрощенном виде) выглядит так:

$$\text{NTLMv2 Response} = \text{HMAC_MD5}(\text{Промежуточный_ключ}, \text{ServerChallenge} + \text{ClientChallengeBlob})$$

Именно этот NTLMv2 Response статья в общем смысле и называет "NTLMv2 хэшем" или "ответом на вызов".

Шаг 3: Расчет сессионного ключа

Наконец, для вычисления сессионного ключа (Session Key или просто Key), который будет использоваться для подписи трафика (MIC), используется снова промежуточный ключ из Шага 1 и ответ клиента из Шага 2. Формула, которую вы привели, становится абсолютно логичной, если подставить в нее наши термины:

$$\text{Key} = \text{HMAC_MD5}(\text{Промежуточный_ключ}, \text{NTLMv2 Response} + \text{ServerChallenge})$$

Таким образом, чтобы разрешить путаницу в контексте алгоритма расчета сессионного ключа:

Термин "NTLMv2 Hash" в формуле $\text{Key} = \text{HMAC_MD5}(\text{NTLMv2 Hash}, \dots)$ означает не ответ клиента, а промежуточный хэш, вычисленный по формуле $\text{HMAC_MD5}(\text{NT Hash}, \text{Uppercase}(\text{Username}) + \text{UserDomain})$.

Автор статьи, для краткости, использовал один и тот же термин для обозначения как промежуточного ключа, так и финального ответа клиента, что и вызвало у вас справедливые вопросы. Ваше глубокое погружение в детали заслуживает уважения! Надеюсь, это объяснение помогло расставить всё по своим местам.

[Tak10](#)
